

Desafio técnico - Processamento de documentos fiscais

Você irá implementar uma API simplificada para solicitar e acompanhar a emissão de um documento fiscal para uma ordem de serviço.

O objetivo não é reproduzir regras fiscais reais. Queremos avaliar como você estrutura uma aplicação C#/.NET, toma decisões técnicas, trata falhas de integração e comunica suas escolhas.

Contexto

Um ERP de oficinas mecânicas precisa emitir documentos fiscais após a conclusão de uma ordem de serviço.

A emissão depende de uma integradora externa, que pode apresentar lentidão, indisponibilidade e respostas inconsistentes. Uma mesma solicitação não pode gerar duas emissões.

Requisitos funcionais

Implemente uma API com os recursos abaixo.

1. Solicitar emissão

Endpoint sugerido: POST /api/v1/fiscal-documents

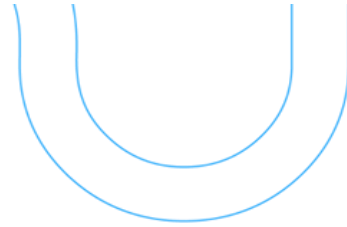
Exemplo de requisição:

```
{
  "workOrderId": "OS-12345",
  "customerDocument": "12345678901",
  "amount": 250.75,
  "municipalityCode": "3550308"
}
```

Comportamento esperado:

- Validar os dados obrigatórios.





- Criar uma solicitação de emissão.
- Impedir processamento duplicado para a mesma ordem de serviço.
- Retornar um identificador da solicitação e o status inicial.
- A emissão não precisa ocorrer obrigatoriamente dentro da requisição HTTP.

2. Consultar status da solicitação

Endpoint sugerido: GET /api/v1/fiscal-documents/{id}

Comportamento esperado:

- Retornar o status atual da solicitação (por exemplo: pendente, em processamento, emitida, falha).
- Quando emitida, retornar os dados do documento fiscal (por exemplo: número ou protocolo).
- Em caso de falha, informar o motivo e se ainda haverá nova tentativa.
- Retornar erro adequado quando a solicitação não existir.

3. Integração externa simulada

Crie uma abstração para uma integradora fiscal. A implementação simulada deve permitir representar pelo menos estes cenários:

- emissão aprovada;
- falha temporária ou timeout;
- falha definitiva;
- resposta demorada.

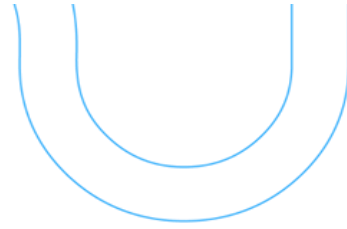
Não é necessário integrar com uma API real.

4. Processamento e recuperação

Explique e implemente, na medida adequada ao tempo disponível:

- como evita duplicidade;
- como trata uma falha temporária;
- como diferencia falhas temporárias de definitivas;





- como uma emissão que ficou pendente poderia ser reprocessada;
- como o sistema registra informações úteis para suporte e auditoria.

Requisitos técnicos

- .NET 8 ou superior (C#);
- dotnet CLI, com dependências gerenciadas via NuGet;
- banco relacional (por exemplo, com Entity Framework Core ou Dapper) ou solução em memória, justificando a escolha;
- testes automatizados para os fluxos principais (xUnit, NUnit ou MSTest);
- tratamento consistente de erros;
- documentação para executar o projeto.

Uso de IA

O uso de IA é permitido e esperado como ferramenta de produtividade. No README, inclua uma seção curta com:

- quais ferramentas de IA utilizou;
- em quais partes elas ajudaram;
- quais sugestões foram alteradas ou descartadas;
- como você validou o código produzido ou sugerido.

Entrega

Tempo sugerido: até 6 horas. Envie por e-mail para marcos-miranda@ultracar.com.br e patricia-magi@ultracar.com.br:

- link para um repositório GitHub;
- instruções de execução;
- README com decisões e trade-offs;
- uma breve lista de melhorias que faria com mais tempo.

Na apresentação, pediremos que você execute e demonstre a solução e explique as principais decisões.

